



Reusing business components and objects for modeling business systems: The influence of decomposition characteristics and analyst experience



Atish P. Sinha^{a,*}, Hemant Jain^b

^a Lubar School of Business, University of Wisconsin-Milwaukee, Milwaukee, WI 53201-0742, United States

^b College of Business, University of Tennessee Chattanooga, Chattanooga, TN 37403, United States

ARTICLE INFO

Article history:

Received 20 May 2015

Revised 8 July 2016

Accepted 24 July 2016

Available online 26 July 2016

Keywords:

Conceptual modeling

Cognitive fit

Chunking

Analyst experience

Software reuse

Systems development

ABSTRACT

Component-based development (CBD) relies on the use of pre-fabricated business components to develop new application systems, rather than developing them from scratch. It provides an attractive alternative to more established development methods such as object-oriented analysis and design (OOAD). Given the growing demands for using agile methods for software development, we examine if systems analysts are more effective at modeling business systems by reusing business components than by reusing objects. We also examine whether the influence of component or object reuse on modeling performance is moderated by prior experience in systems analysis and design. We evaluate the representational constructs of the two based on a set of decomposition characteristics, and postulate hypotheses comparing the two based on theories in cognitive psychology and human factors. We find that models generated by reusing business components are of higher accuracy than those developed by reusing objects. An interesting finding of our study is that IT professionals who are less experienced in systems analysis and design perform on par with experienced professionals, when modeling business systems by reusing components. We argue that the decomposition characteristics of components—with respect to granularity, quality, and focus—enable less experienced analysts to perform on par with more experienced analysts.

© 2016 Elsevier Inc. All rights reserved.

1. Introduction

Universal standards for technologies to build software systems using components have emerged, making it feasible to use the component-based development (CBD) method as a mainstream approach for developing business systems (Heineman and Councill 2001; Herzum and Sims 2000). In this environment, the web services standards (Zhang 2007) are generally used as a platform-independent way of delivering the functionality of components through the Internet. However, component functionalities can also be delivered through other interface standards. A service is a coarse-grained, discoverable entity that interacts with applications and other services (Elfatraty 2007). In essence, components encapsulate business functionality while services offer the means for delivering that functionality (Levi and Arsanjani 2002; Crnkovic et al. 2011). A service-oriented architecture (Erl 2004) provides an approach for architecting such applications. Thus, an agile approach for the design of composite applications, which are

assembled by piecing together the functional units of business logic from the components and services, are becoming increasingly popular (Haines and Rothenberger 2010).

Components hold an esteemed place in software engineering history since componentization is a fundamental engineering principle (Crnkovic et al. 2011). Heineman and Councill (2001, p. 7) define a *software component* as “a software element that can be independently deployed and composed without modifications according to a composition standard”. A component is a unit of composition with contractually specified interfaces, and can be deployed independently and subjected to composition by third parties (Szyperski 2002). Components can therefore be independently produced, acquired, and deployed to form a functioning system. Although components originated as implementation units, their use has gradually spread to the realm of analysis and design through the concept of business components (Janssen and Wagenaar 2006). A business component models and implements business logic, rules, and constraints that are typical and recurring notions in a business domain (Barbier and Atkinson 2003). As a “traditional” component, a business component is a self-contained software entity with well-defined interaction points (called interfaces) facilitating access to and execution of a coherent package of functional-

* Corresponding author.

E-mail addresses: sinha@uwm.edu (A.P. Sinha), Hemant-Jain@utc.edu (H. Jain).

ity. “Coherent” means that the business logic, rules, and constraints are separated into highly cohesive and loosely coupled elements.

The focus of this paper is on business components that are developed commercially for assembling business applications without reference to the underlying technology (Vitharana et al. 2003b). Business components migrate through the software development cycle and are connected via a continuum with technical components that encapsulate software facilities such as primary data structures, generic algorithms, common services (e.g. date/time stamping, graphics, user interface, network communication, data storage) (Barbier and Atkinson 2003). Business components can be delivered and installed separately, and their interfaces are independently accessible at runtime (de Cesare et al. 2006). Each component can be combined and composed with other components to provide greater functionality. Firms can develop customized applications to support an array of enterprise business processes by assembling these components (Vitharana et al. 2003b).

Component-based development, using prefabricated business components, is a relatively new paradigm for software development, compared to established approaches such as object-oriented analysis and design (OOAD) (Booch 1994; Booch et al. 1999; Larman 2002) and structured analysis and design (SAD) (Gane and Sarson 1979). The main motivation behind using CBD is *reusability* (Sugumaran and Storey 2003). Components, which could be available in-house or purchased from third parties, are designed to be composable, to have plug-and-play functionality, to be replaceable, and, therefore, to reduce system development time significantly (Lau and Wang 2007). Business components are increasingly stored in specially designed repositories and used for analysis, design and building of applications (Vitharana, et al. 2012, Koschmider et al. 2014). Additionally, CBD enhances the agility of application systems by allowing individual components to be replaced to accommodate changing business requirements. CBD, therefore, provides an attractive alternative to developing business applications from scratch by using existing components or acquiring *off-the-shelf* components, and plugging them together.

While CBD is growing in popularity, OOAD is still being widely used for modeling and design of software systems. However, rapid advances in internet technologies are placing new demands on how applications are developed. IT organizations are being increasingly tasked with delivering business solutions through distributed, web-based applications. Well-established systems development methods are slowly giving way to agile methods that exploit the reuse and assembly of a variety of components, services, and widgets. Mahmood et al. (2014) advocate employing reuse during the early stages of the software development cycle. Sinha and Jain (2013) found that IT professionals perceive components to be much easier to reuse than objects.

Given the growing demands for using CBD to address the challenges in a rapidly changing environment, it is important to examine if component reuse is indeed more effective for modeling business systems than object reuse. Effectiveness here implies how well systems analysts and designers can: i) identify the appropriate compositional units (components or objects¹) from an existing library for developing a system, and ii) assemble those units together to develop a *conceptual model* of the system.

Our study addresses the gaps in the literature by empirically examining the effectiveness of objects and components for reuse-based modeling, within a research framework rooted in cognitive psychology and human factors. Specifically, we address the following research questions: i) Given a library of reusable assets, are

IT professionals more effective in creating conceptual models of business systems by reusing components or by reusing objects? ii) Does prior experience in systems analysis and design moderate the quality of the models developed by IT professionals through component/object reuse?

To address the research questions above, we conducted an experiment in which two groups of industry professionals completed tasks related to conceptual modeling of a business system, with one group reusing objects and the other reusing components, both from the same domain. In Section 2 we provide the technical background for our study and in Section 3 we present the research model. Sections 4 and 5 present the literature review and hypotheses, respectively. Section 6 describes the experimental design and methodology and Section 7 presents the experimental results. We discuss the implications of the results in Section 8 and the limitations of our study in Section 9. Section 10 concludes the paper and provides future directions.

2. Technical background

Proponents of OOAD argue that objects are a natural way to building software systems because they reflect the real-world entities of a domain. This facilitates organization of IT system elements and relating them to the real-world entities. However, business users and analysts generally think of a business system as a collection of process elements arranged to support unique organizational processes. This includes business logic, rules, and constraints that govern the process. In many cases, a process element supporting a single discrete business function needs to span multiple objects. For example, a process element representing discrete business functionality, such as “verify order,” needs to use multiple object classes like Customer, Order, and Item, and needs to include business logic, rules, and constraints to verify the order. An “order verification” business component can encapsulate all of the above functionality required to support the “verify order” process element. Hence, business components address one of the criticisms of OOAD, the relatively low level of abstraction of objects and the lack of consistent connection to business-oriented concepts—specifically to the elements of business processes—thereby adversely affecting application development time and complexity (Brown 2000).

The proponents of CBD argue that it reduces the skill requirements of analysts. In other words, it is not necessary for a developer to acquire significant experience in systems analysis and design before he or she has to model a business system through component reuse (Sprott 2000). On the other hand, prior research has shown that experience plays a significant role in modeling systems using OOAD (Agarwal et al. 1996b; Khatri et al. 2006). However, the role of experience has not been examined in the context of reuse, which is the focus of this study.

2.1. Class and component diagrams

A *class diagram* is the most common type of diagram used for modeling object-oriented (OO) systems. It shows the static structure of a system in terms of object classes, their attributes and operations, and their relationships with one another. Each class represents a set of objects that share a common structure and a common behavior. Operations describe the behavior of a class.

A *component diagram* represents the static view of a system by depicting the appropriate components and their relationships. An important construct in component diagrams is the interface. An *interface* is a collection of operations that are used to specify a service provided by the component (Booch et al. 1999). A component can have multiple interfaces, thus providing multiple services. Interfaces of such components are business-oriented and contain operations to support discrete activities in business processes.

¹ We use the terms “class” and “object” interchangeably, though, strictly speaking, an object is an instance of a class.

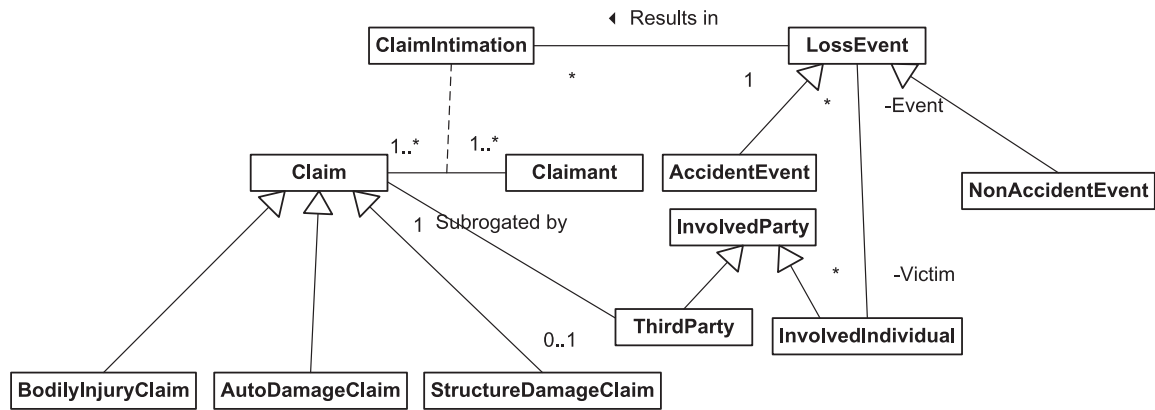


Fig. 1. Partial class diagram for the auto insurance system.

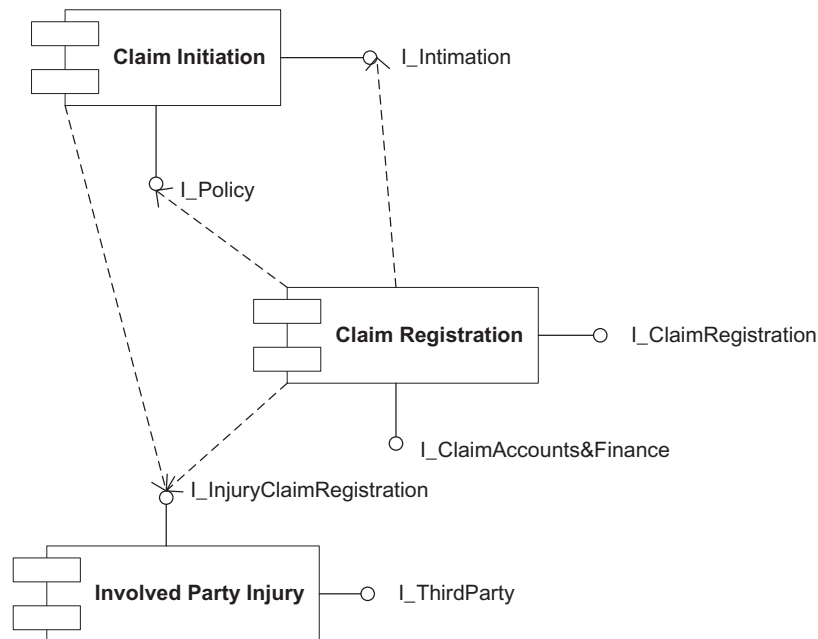


Fig. 2. Partial component diagram for the auto insurance system.

Fig. 1 shows a partial class diagram in the insurance claims domain with a class called **Claim** and three subclasses: **BodyInjuryClaim**, **AutoDamageClaim**, and **StructureDamageClaim**. The behavior of **Claim** is represented by a number of operations such as **updateClaimStatus**, **checkMandatoryClaimDetails**, and **createClaimRecord** (see Appendix B). External objects interact with the **Claim** object through these public operations. Subclasses such as **BodyInjuryClaim** inherit all the operations of the **Claim** class, and have their own unique operations; for example, **BodyInjuryClaim** has two operations: **createBodyInjuryClaimRecord** and **getBodyInjuryClaimDetails**.

The **Claim** class participates in an association relationship with the **Claimant** class. The relationship represents the fact that a claimant files one or more claims, and each claim has one or more claimants. **ClaimIntimation** is an *association class*, implying that it originated from the relationship, but is modeled as a class because it has features of its own. The important thing to note is that it is the structural relationships that provide the glue for combining the classes in a class diagram.

Fig. 2 shows a partial component diagram in the same domain with three business components: **Claim Initiation**, **Claim Registration**, and **Involved Party Injury**. The claim initiation component supports the process element of initiating a claim, which

involves intimation of incidence, collection of incidence details, and verification of policy. This component has two interfaces, **I_Intimation** and **I_Policy**, each specifying a service that can be used by other components. For example, **I_Intimation** provides claim intimation service through three operations: **initiateIntimation**, **getIntimationDetails**, and **changeClaimantEstimates**. The dependency relationship from **Claim Registration** to **I_Intimation** (shown as a dashed arrow) implies that the former uses the intimation service of the **Claim Initiation** component. For component diagrams, it is the services specified in the interfaces that are used to bind the components together.

2.2. Objects versus components

We find that while components and classes share similarities (encapsulation, maintainability, reusability, etc.), there are major differences. Table 1 summarizes the differences between business components and objects. Components tend to represent business processes whereas objects tend to represent real-world entities. They are at different levels of abstraction. The grain size of a component is generally larger because it packages multiple classes. But the main distinction we want to emphasize is that a business component is *service-oriented*—it is designed to provide services

Table 1
Comparison of objects and business components.

Characteristics	Business components	Objects
Representation	Business processes	Real-world entities
Abstraction	High level	Low level
Focus	Services	Entities & concepts
Granularity	Relatively high	Relatively low

supporting distinct activities in a business process—unlike an object, whose focus typically is on a problem domain entity or concept. The service focus of a component provides it with a significant reusability advantage compared to a class. As Booch et al. (1999) note:

“... a component is *replaceable*. A component is substitutable – it is possible to replace a component with another that conforms to the same interfaces... a component is *part of a system*... a given component collaborates with other components... A component is logically and physically cohesive and thus denotes a meaningful structural and/or behavioral chunk of a larger system. A component may be reused across many systems. Therefore, a component represents a fundamental building block on which systems can be designed and composed.” (p. 349)

This can be illustrated by an example from the insurance claim domain; here the **Claim Registration** component encapsulates the functionality for registering a claim. In the OO representation, this functionality is spread across different classes—**Claim**, **Claimant**, **ClaimExpense**, **SupportingDocuments**, etc.—and their relationships with one another. The **Claim Registration** component not only packages these related (highly coupled) classes, but also encapsulates the relationships they participate in. It exposes its functionality as services through interfaces such as **I_ClaimRegistration** and **I_ClaimAccounts&Finance**, which are at a higher level of abstraction than the base level operations in the constituent classes. This allows the developer to focus on a service as a whole, as opposed to identifying and linking different operations that are distributed across multiple classes.

2.3. Component design and development process

Considering the characteristics of business components described above, component design should strive for tight cohesion within a component and loose coupling between components (Chang et al. 2005; Jang et al. 2003; Lee et al. 2001). Cohesion refers to the strength of relationships between elements (classes) in a component, while coupling refers to the extent to which classes in one component relate to classes in other component. The ideal scenario is one in which the cohesion within a component is maximized and the coupling between components is minimized, and objectives such as ease of assembly, maintainability, and development cost are also optimized. Vitharana et al. (2004) proposed a strategy-based approach for designing reusable business components. This approach was further enhanced for service design and implemented in a CASE tool (Jain et al. 2007). Erl et al. (2009) proposed a web service contract design and versioning methodology. Birkmeier and Overhage (2009) provide an excellent review of component identification and design approaches.

In this study, the subjects used the UML notation for developing the conceptual object and component models. Note, however, that other tools such as ArchiMate could also be used as an alternative for developing the conceptual models (Jonkers et al. 2011). The method implemented by Jain et al. 2007 in a CASE tool was used to design the components/services. The approach takes

an analysis level object model, representing a business domain, as the input and generates potential component/service designs, in which the classes in the object model are grouped into appropriate components/services based on high cohesion and low coupling between the classes. Next, employing a multiobjective genetic algorithm, it optimizes a set of managerial objectives, such as ease of assembly, maintainability, and cost effectiveness, to identify a set of non-dominated solutions. Finally, a preferred component design solution is selected from this set.

The components designed for the insurance claim domain using the above approach were fabricated for reuse by a large global IT firm. The creation of services does require implementation of constituent objects, which were done in Java. An IDE, like the one provided by Oracle or provided in IBM WebSphere (IBM 2016), was then used. The availability of these tools made the process of developing components/services from constituent objects quite easy.

3. Research model

The focus of this study is on comparing the modeling performance of analysts using a library of reusable components with that of analysts using a library of reusable objects. Our study can therefore be framed within the context of software reuse research (Frakes and Kang 2005), with the reusable assets at the analyst's disposal being components or objects. The analyst's job is to understand the requirements of the business application, identify the correct set of reusable assets, and assemble them properly to develop the system. Fig. 3 depicts the reuse-oriented modeling process.

In the previous section, we saw that components and objects are at different levels of abstraction and granularity, and that their emphases are different. A component integrates multiple classes, business logic, rules, and constraints; its interfaces specify the services provided by the component. Both the component and the interface service are at a higher level of abstraction than the corresponding classes and operations in the OO representation. Therefore, the number of components needed to develop the system would be in general less than the number of objects.

A component is a structural and behavioral *chunk* of a larger system (Booch et al. 1999). Research in cognitive psychology suggests that experts avail of chunks representing functional units in their domains (Chase and Simon 1973; Larkin et al. 1980). In particular, the chunking phenomenon has been observed in the programming domain (Adelson 1981; McKeithen et al. 1981; Shneiderman 1976). A chunk represents a familiar unit of experience. While a component corresponds to a chunk in CBD, a class corresponds to a chunk in OOAD.

Each of the interfaces of a component can be considered to be a behavioral chunk geared toward providing a specific business service. In contrast, the OO representation does not offer such service-oriented constructs, thus placing the onus on the modelers themselves to mentally group together lower-level operations to support the business activities of the scenario.

As noted above, both the class and component libraries used in this study were developed by a global IT firm for the insurance claims domain. Each business component was fabricated to provide access to a discrete, coherent package of functionality. To deliver coherent functionality, components were designed so that they are highly cohesive. In the cognitive psychology literature, such a cohesive and meaningful unit of information is referred to as a chunk. The **Claim Registration** component, for example, is a structural and behavioral chunk that is geared toward providing a coherent functionality, that for validating and registering insurance claim.

The focus of OOAD is on real-world entities and concepts, not on business services. Because it captures a real-world entity or concept, an object is a familiar and meaningful unit of information

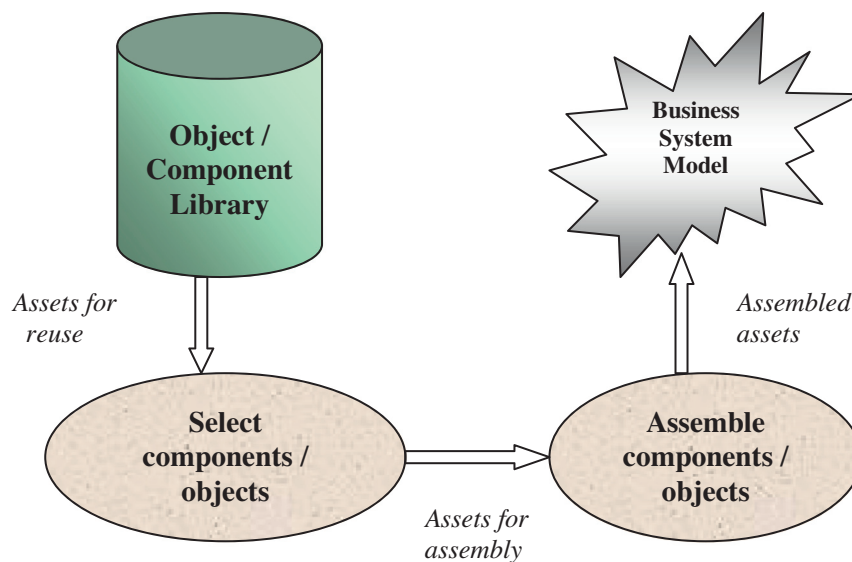


Fig. 3. Reuse-oriented modeling of business systems.

Chunk Representation

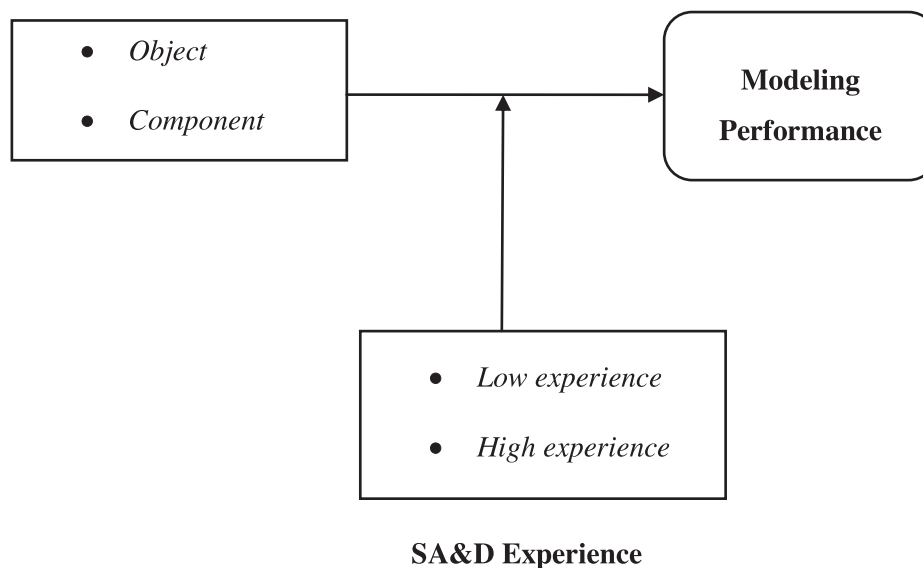


Fig. 4. Research model.

for modelers employing OOAD. Therefore, an object represents a chunk in OOAD. But the size of the chunk represented by an object is much smaller than that of a component, because the entity or concept it represents is at a much lower level of abstraction than a business process. As we will argue later, the number and size of chunks have a significant influence on the effectiveness of modeling business systems.

Fig. 4 presents our research model. The antecedent of modeling performance is the underlying representation of chunks in the two modeling methods. In this study, the two representational constructs for chunk are *object* and *component*. We examine the inherent characteristics of these constructs to postulate a set of theory-driven hypotheses on modeling performance. The characteristics that we examine are *decomposition granularity*, *decomposition quality*, and *focus*.

Fig. 5 graphically depicts the characteristics of the two constructs along the three dimensions: *decomposition granularity*, *decomposition quality*, and *focus*. We have discussed “granularity” and “focus” in detail above. In the context of conceptual modeling, prior research (see, for example, Burton-Jones and Meso 2008; Wand and Weber 1990) has examined the role of decomposition quality, which refers to how well an analyst has broken down a problem into its constituent parts or subsystems. We define the decomposition quality of a construct based on two criteria: *coupling* and *distribution of functionality*. We consider a reusable asset that is loosely coupled with other artifacts and that supports a cohesive, complete business function to have high decomposition quality. As discussed before, a component, which is fabricated based on the criteria of loose coupling and strong cohesion, caters to a discrete business function. In contrast, a business function,

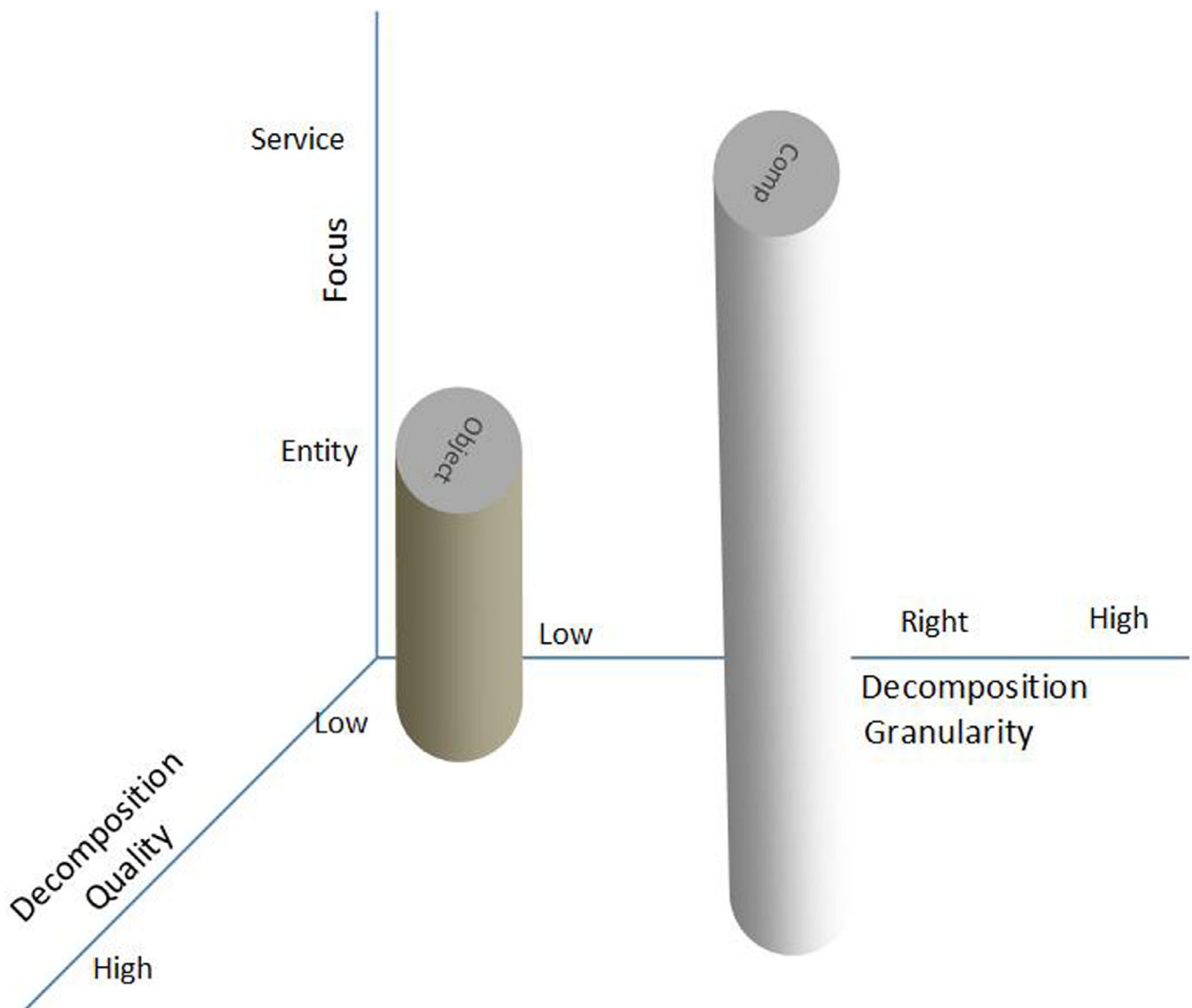


Fig. 5. Characteristics of object and component.

which cannot be supported by one object, is distributed across multiple, tightly-coupled objects.

Because of these reasons, the decomposition quality of components would be of much higher quality than that of objects. Of course, their grain size would also tend to be larger than that of objects. Note, however, that higher grain size alone does not guarantee higher quality. If a large asset, formed by combining multiple objects, is not cohesive enough, or is tightly coupled to other assets, its decomposition quality would be low. For service integration, granularity negatively correlates with reuse; larger-grained services are less likely to be reused (Zimmermann et al. 2005). To be effective for reuse, therefore, components should have the “right” grain size.

4. Literature review

The two representational constructs for chunks, object and component, are quite different in terms of granularity, quality, and focus; these characteristics in turn influence the complexity of the modeling task. Wood (1986) postulates that all tasks consist of three essential components: products, acts, and information cues.

In this study, the product of interest is the business system model generated by an analyst. The acts include the modeling activities the analyst performs in developing the model. The stimuli of the representational constructs (chunks) act as information cues that the analyst attends to and processes while performing the modeling task.

Task complexity depends on the number of distinct acts that have to be performed and on the number of distinct information cues that must be processed (Wood 1986). As discussed below, the service focus of a component enables it to provide an explicit type of information cue by exposing its services through its interfaces; this helps reduce the complexity of the modeling task. Higher granularity does not necessarily imply lower complexity but, as discussed above, if the construct also has the “right” size and higher decomposition quality, then task complexity tends to be relatively lower.

In addition to decomposition granularity, quality, and focus, prior experience in systems analysis and design could have a bearing on conceptual modeling performance. Agarwal et al. (1996b) conducted a study in which they compared the modeling performance of experienced and inexperienced analysts.

The first group consisted of systems analysts and designers with experience in process-oriented modeling, while the second group consisted of subjects with very little process-oriented modeling experience; none of the groups had any OO modeling experience. Both groups were asked to develop OO models for two types of tasks. The researchers found that for both tasks, the experienced analysts performed better than the inexperienced ones. However, at the “structure” subtask level, they did not find any significant differences.

Khatri et al. (2006) examined the effects of IS domain knowledge. They found that such knowledge helps improve performance in all types of conceptual schema understanding tasks. Vessey and Galletta (1991) investigated the effect of cognitive fit between technology and task on problem-solving performance, and then extended the paradigm to include information processing skills. They found that performance improved when specific problem-solving skills were matched to the problem representation and task. When skills were matched to the task alone, performance improved but not to the same extent.

Modelers who have systems analysis and design experience may have developed hierarchically-organized knowledge structures or schemata (Chi et al. 1982) that they could exploit. We contend, therefore, that the influence of the representation used on modeling performance would be contingent on the analyst's systems analysis and design experience. That is, as Fig. 4 shows, the influence of *Modeling Representation on Performance* is moderated by *SA&D Experience*.

The systems analysis task is geared toward understanding and specifying the functional requirements of the system in the form of a *conceptual model*. The focus is on *what* services the system should provide, not on *how* it should provide them. Developing the conceptual model in a reuse-based environment entails that the analysts *select* the appropriate components/objects from the library and *assemble* them in a component/class diagram.

The *selection* process—an activity that sets reuse-based conceptual modeling apart from traditional modeling—is considered to be very important in the CBD approach. Selecting the right components/objects is one of the key challenges (Lorenz and Vlissides 2001; Schneider and Han 2004; Vitharana et al. 2003a). The *assembly* process follows, where the selected components/objects are plugged in to develop the business system (Vitharana et al. 2003a).

The selection and assembly of reusable assets will be facilitated if modelers can easily and quickly draw inferences from the information provided in the representation of the assets in the library. The effectiveness of the inference process depends on the data structures and operators available in the representation to make the inference (Larkin and Simon 1987). The data structures for the object-oriented and component-based representations are available in the class and component libraries, respectively. The productions or inference rules for operating on the data structures are part of the knowledge of OOAD and CBD principles and techniques that the modeler stores in memory. An attention management system determines which part of the data structure the modeler is focusing on, so that the program operators can be triggered (Larkin and Simon 1987).

Selecting the right objects or components entails searching the library for assets that can be reused in a given situation, based on the modeler's knowledge of the specific representation. Attention management is necessary for this search process. Larkin and Simon (1987) contend that a major source of computational inequivalence between two representations is the difference in the search strategies employed. A component is a representation of a discrete business function, and each of its interfaces provides a specific service. It facilitates effective attention management by focusing the modeler's attention on a few cohesive components and their interfaces, each of which provides a set of related operations

to provide a service. In contrast, the focus of the object representation is on problem domain entities and concepts; the operations needed to provide a specific service are typically distributed across several objects, thereby rendering attention management much less effective. Attending to and processing fewer information cues reduces the task complexity (Wood 1986) of component-based modeling vis-à-vis object-oriented modeling.

Our research model (see Fig. 4) depicts the moderating influence of prior analysis and design experience on modeling performance. Several prior studies (e.g., Agarwal et al. 1996b; Khatri et al. 2006; Vessey and Galletta 1991) have examined the role of experience, IS domain knowledge, and skills. Based on experience, experts are able to develop hierarchically-organized schemata (Chi et al. 1982). They are able to activate higher-level schemata that are relevant (LaFrance 1989). Past research has shown that experts are good at automating the low-level details novices are attentive to, focusing instead on larger-sized functional chunks that are relevant for addressing the problem at hand (Buxton 1986).

The systems analysis task entails two important activities: *decomposition* and *recomposition*. Decomposition helps the analyst address the complexity of a problem by breaking it down into smaller, manageable pieces. The analyst then develops solutions to those subproblems and combines them during the recomposition process to arrive at a solution to the overall problem. The objects available in a library for reuse correspond to the smaller pieces resulting from the decomposition process during a systems analysis task. We would like to point out that prior studies investigating modeling effectiveness have dealt with models generated from scratch. In those studies, developing a conceptual class diagram, for instance, entailed creating the classes, attributes, and operations, and then connecting the classes through relationships. In contrast, we are investigating conceptual model development from a reuse standpoint, where the needed objects have already been designed and stored in a library.

Among the advantages claimed by CBD proponents is the one that a system developed from components reduces skill requirements (Due 2000; Sprott 2000). As Sprott (2000) notes, “The parts of a componentized software product may be constructed by potentially different manufacturers and then brought together to be assembled very rapidly. The assemblers do not need to be experts in the internals of each assembled component.”

A component represents a separate entity that encapsulates a behavioral chunk of a business process, which matches the process-oriented nature of human problem solving (Agarwal et al. 1996a; Papert 1980). The coarse granularity, well-defined scope, and internal cohesion of a component make it easier to understand, manage, and upgrade the component, thereby requiring less experience (Sprott 2000). Another aspect to consider is that a component delivers services, which are what the analyst uses. In general, useful services are designed to operate at an abstraction level that is easily understandable to users and analysts alike. For components, the understanding is facilitated through encapsulation and information hiding features (Vitharana and Jain 2000). In contrast, objects encapsulate data but not control or computation. They are not compositional (Lau and Wang 2007). In CBD, encapsulation occurs at every level of composition, with every composite encapsulated into an interface. As Lau and Wang (2007) note:

“... interface is all we need to know about the composite in order to use it for further composition. This means that we can encapsulate much larger composites at each step and, by so doing, we are able to subsequently compose much larger composites without regard to their size or complexity.”

This makes it easier for the inexperienced analyst to create the conceptual model using CBD.

5. Hypotheses

After the assets are selected for reuse, they are assembled together to form the model of the business system (see Fig. 3). For the assembly process to take place, the selected objects/components need to be first included in a class/component diagram. In a component diagram, a component provides explicit cues for dependency connections from other components by explicitly exposing its services through its interfaces. In contrast, an object in a class diagram does not provide such explicit cues; connections to other objects are implicit in the structural relationships that exist in the problem domain. Hence, we can claim that the connections provided in the component data structure are more effective for asset assembly than the object data structure. As Larkin and Simon (1987, pp. 67–68) state, “the nature of attention management depends crucially on the linkages provided in the data structure.”

We therefore find that, from the standpoint of granularity, decomposition quality, and focus, a component is computationally efficient, modular, and service-focused. The information emphasized in both the component representation and the systems analysis task is service-oriented, resulting in *cognitive fit* (Vessey 1991). Cognitive fit allows the analyst to develop a consistent mental representation to operate on and subsequently exhibit superior conceptual modeling performance. Also, because of the right granularity and high decomposition quality, the component data structure enables better attention management and lower complexity during the search process. We therefore posit the following hypothesis:

H1: Systems analysts develop more accurate conceptual models of business systems by reusing components than by reusing objects.

Experience with decomposition-recomposition activities helps an experienced analyst learn to perform the low-level details automatically and concentrate on larger chunks of the problem. So while we can expect an inexperienced analyst to focus on the “low-level” objects—missing the forest for the trees—an experienced analyst is able to mentally activate the higher-level schemata that she possesses to recognize large *chunks of objects* at a higher level of abstraction. Much like a chess master who is able to recognize larger sized chunks on a chess board and take appropriate actions to deal with the situation (Chase and Simon 1973), an experienced analyst can exploit hierarchically-organized knowledge structures to form a mental representation consisting of cohesive chunks—which hide the low-level operations of and coupling among objects—and generate a superior conceptual model. We therefore posit the following hypothesis:

H2: Analysts with higher experience in systems analysis and design techniques develop more accurate conceptual models of business systems when reusing objects.

As discussed before, because of their decomposition/recomposition skills, experienced analysts can identify relevant objects and then assemble them to arrive at a solution more effectively than less experienced analysts. That is something that inexperienced analysts, who do not possess such skills, are not adept at. The picture is markedly different for CBD. Inexperienced analysts do not have to deal with lower-level objects anymore. Instead, what they find in the repository is components that encapsulate service-oriented composites at the interfaces. The hierarchically-organized decomposition/recomposition schemata that provided experienced analysts with a significant advantage in identifying chunks of objects and assembling them are no longer that necessary because components replicate those mental chunks physically in external memory (repository). So the handicap that inexperienced analysts face with respect to decomposi-

tion/recomposition knowledge structures is compensated for by the high-level chunking of reusable service-oriented components available in external memory—the repository. We therefore posit that:

H3: Analysts with higher experience in systems analysis and design techniques do not develop more accurate conceptual models of business systems when reusing components.

6. Experimental design and procedure

The research hypotheses stated above were tested in a controlled experimental study. In empirical software engineering, “experiment” is usually the adopted methodology. One of the major advantages of experimental research is that it helps us to achieve control through randomization and active manipulation of the independent variables (Kerlinger 1986). In this section, we describe the experimental design and procedure we employed to test the research hypotheses and to achieve the desired control.

6.1. Independent and dependent variables

Our research model (see Fig. 4) examines the influence of the object and component reuse on conceptual modeling performance. The dependent variable for examining those hypotheses, therefore, is *Performance* (details of operationalization of this variable are given below). The independent variable we wish to manipulate is *Chunk Representation*. We use this variable to define two groups (*Object* and *Component*), one in which modelers develop a business system by reusing objects and another in which modelers do the same by reusing components.

6.2. Experimental design

A one-way analysis of variance would ordinarily be an appropriate design to adopt for such an experimental study, where the measurement of the independent variable (factor) under consideration divides the sample into two or more groups. Any systematic bias between the two groups (hereafter referred to as OO for the group reusing objects, and COMP for that reusing components), with respect to performance, can be effectively addressed by randomly assigning subjects to the two groups. However, even after eliminating systematic bias between the groups, there could still be some *within group* or *error variance* (Stevens 1992). In this study, within group variability could arise because of prior experience or domain knowledge. In such situations, *analysis of covariance* is an appropriate research design to use. It is an analysis of variance method that “tests the significance of the differences among means of experimental groups after taking into account initial differences among the groups and the correlation of the initial measures and the dependent variable measures” (Kerlinger 1986, p. 339).

6.3. Covariates

Any variable (*X*) that is significantly correlated with the dependent variable (*Y*) is a potential *covariate*. The potential covariates in this study are SA&D Experience, OOA&D Experience, Programming Experience, OO Programming Experience, and Insurance Knowledge (see Table 2). We use these variables as covariates or control variables to factor out any differential effects that these variables could have on performance. Using an analysis of covariance design helps us to adjust the means of the OO and COMP groups to what they would be if both groups started out equally on the covariates.

Table 2
Analysis of covariance experimental design.

Object representation						Component representation					
X_1	X_2	X_3	X_4	X_5	Y	X_1	X_2	X_3	X_4	X_5	Y
SADExp	OOAExp	ProgExp	OOProgExp	InsKnow	Performance	SADExp	OOAExp	ProgExp	OOProgExp	InsKnow	Performance

Table 3
Demographic information.

	OO (N = 37)		COMP (N = 36)		Total (N = 73)	
	Mean	Standard deviation	Mean	Standard deviation	Mean	Standard deviation
Age [†]	26.95	3.496	27.00	3.024	26.97	3.249
SA&D Experience	2.30	0.996	2.69	0.980	2.49	1.002
OOA&D Experience	1.57	0.801	1.53	0.609	1.55	0.708
Prog Experience	3.32	0.784	3.61	1.001	3.47	0.903
OO Prog Experience	2.24	0.760	2.06	0.924	2.15	0.844

[†] In years.

6.4. Procedure

Prior to conducting the experiment, a pilot study was conducted with four subjects trained in both OOAD and CBD methods. Two of them used a library of reusable objects while the other two used a library of components. After completing the experimental tasks, the subjects were debriefed on whether they had any difficulties with respect to understanding the OOAD and CBD training materials, task instructions, domain problem description and scenarios, class/component descriptions, and operation/interface descriptions. They reported that they were satisfied with the training, task instructions, class/component and operation/interface descriptions, and problem description and scenarios. One subject suggested that a simple case be provided for practice with class and component diagrams exercises prior to the experiment. Based on the suggestion, we included practice exercises for completion prior to the experiment.

All the subjects who participated in the experiment worked for a major international IT consulting firm. We sent a request to the employees in the firm to participate in a study examining the effectiveness of the CBD vis-à-vis OOAD for modeling business systems. We targeted systems/business analysts with one to five years of systems development experience, zero to minimal experience in OOAD and CBD methods, and zero to minimal experience in auto insurance claim systems. They were also told that if they participate, they would have to complete a set of tasks based on a business case and accompanying scenarios. As an incentive, potential participants were told that they would receive free training in object-oriented and component-based development.

We specifically targeted analysts with zero or minimal experience in OOAD and CBD because we wanted to examine how effective they were at reusing objects/components for conceptual modeling without prior experience in applying those methods. For firms that want to promote reusability, it is realistic to assume that they would be looking at this kind of target pool, especially given that very few analysts have experience with the emerging CBD approach.

In all, 76 IT professionals volunteered to participate in the study. None of them had prior experience in CBD, while most of them had very little or no experience in OOAD. Each of them attended a training session on object-oriented and component-based software development concepts. The training session lasted for 2 h 10 min, with a break in the middle. Each participant was then given practice exercises to work on, using both object-oriented and component-based approaches. The practice session lasted for 1 h 10 min, after which a lunch break was given. Three participants

dropped out after the training and practice sessions, leaving us with a pool of 73 subjects.

The subjects were randomly assigned to two groups of roughly equal size, with 37 subjects assigned to the OO group and 36 to the COMP group. Each subject was given a maximum time of 2 h 15 min to complete the experimental tasks. Each subject completed a background survey before participating in the experiment.

Table 3 provides the demographic information of the subjects. Except for Age, which was recorded in number of years, all the attributes were measured on a Likert scale of 1–5, with 1 indicating no experience, 3 indicating some experience, and 5 indicating a lot of experience. SA&D Experience refers to the experience in structured systems analysis and design techniques (e.g., data flow diagrams), while OOA&D Experience refers to experience in object-oriented analysis and design techniques (e.g., UML class diagrams). The subjects also reported their general computer programming experience (Prog Experience) in languages such as COBOL and Basic, as well as their object-oriented programming experience (OO Prog Experience) in languages such as Java. Finally, they also rated their knowledge of the automobile insurance domain (Insurance Knowledge), which they had almost no knowledge of (mean = 1.27). Out of the 73 subjects who participated, 59 reported absolutely no knowledge of the insurance domain; only 14 had very limited knowledge. The mean scores on the experience-related attributes for the two groups are quite close, indicating that the desired effects of randomization were attained.

Each subject was given the following materials: an information packet consisting of a description of the study and the experimental procedure; the ABC Claims Processing System case along with three scenarios (see Appendix A); and the experimental tasks. The information packets for the two groups differed only in the specification of the experimental tasks, with the OO group asked to model the insurance system using the OOAD method, and the COMP group asked to do the same using the CBD method.

For the first experimental task, subjects were asked to select the relevant classes or components—for supporting the requirements described in the business case and the scenarios—from an existing library. For the second task, OO subjects were asked to develop a class diagram for the claims processing system, showing the relationships among the classes selected. COMP subjects had to draw a component diagram showing the relationships (dependencies) among the components selected and their interfaces.

In addition to the information packet, each subject was given a class or component report (library), depending on which group he or she was assigned to. OO subjects received a Claims System Class Report which provided a description of each of the

available domain classes, including the functions performed and the roles played by the class, and a description of its operations (see Appendix B for examples). Similarly, each COMP subject was given a Claims System Component Report which provided a description of each of the available components, including the functions performed and the roles played by the component, and a description of its public interfaces and of the exposed operations in each interface (see Appendix C for examples).

6.5. Solution evaluation

We measured performance in conceptual model (CM) development by generating two scores: objective (CMObj) and subjective (CMSub). Two raters, each of whom had over ten years of experience in object-oriented modeling, and over five years of experience in component-based modeling, independently assigned subjective scores for the solutions generated by the subjects with respect to the quality of CM. A subjective score reflects the “overall quality” of a solution for a given task, and can be looked upon as an alternative to the objective measure. While an objective score assesses each part (e.g., classes, relationships, multiplicities) separately and then combines those assessments, a subjective score reflects the notion of correctness on a holistic basis (Agarwal et al. 1996a), as seen through the eyes of an expert.

To derive the objective score for the conceptual model (CMObj), we first considered the number of classes/components a subject correctly selected, as well as the number incorrectly selected. Subjects were rewarded for selecting the right classes/components and penalized for selecting the wrong ones, in proportion to the total number of classes/components in the library. For the OO solution, the score for classes is given by:

$$\frac{\text{Number of correct classes}}{38} + \frac{10 - \text{Number of incorrect classes}}{38}$$

where 38 is the total number of classes in the report, out of which 10 are incorrect. If someone identified the 28 correct classes, and none of the 10 incorrect ones, he or she would receive a perfect score of 1 on this part. As examples of incorrect class, consider **SalvageItem**, **SalvageBidder**, and **SalvageBuyer** in the class library. These classes deal with bidding for and purchasing salvage items, which are not part of the requirements for the insurance claims case the subjects were asked to model. For the COMP solution, the corresponding score is given by:

$$\frac{\text{Number of correct components}}{7} + \frac{2 - \text{Number of incorrect components}}{7}$$

An example of an incorrect component is **Salvage**, which deals with salvage items, bids, and purchases. Classes and components that are not needed for the given modeling task are considered to be incorrect with respect to the score. The second part of the CM score corresponded to the relationships among the classes/components. For the OO solution, the total number of correct relationships is 34, while that for the COMP solution is 9. For the relationship score, we considered the proportion of relationships correctly identified.

Finally, the third part of the CM score dealt with the number of correct multiplicities and supertypes (total of 29) in the required relationships for the OO case, and the number of times (total of 9) the directions for the required dependency relationships were correctly shown for the COMP case.

The CMObj score, therefore, consists of three parts: i) class/component, ii) relationships, and iii) multiplicities/directions. Each part was weighted in terms of importance to the aggregate score by two experts; the weights were 5, 3, and 2 for the first,

second, and third part, respectively. The formula for computing the aggregate score is therefore:

$$\begin{aligned} \text{CMObj} = & 5 * \left(\frac{\text{Number of correct classes}}{38} \right. \\ & \left. + \frac{10 - \text{Number of incorrect classes}}{38} \right) \\ & + 3 * \frac{\text{Number of correct relationships}}{34} \\ & + 2 * \frac{\text{Number of correct multiplicities/supertypes}}{29} \end{aligned}$$

for the OO solution and

$$\begin{aligned} \text{CMObj} = & 5 * \left(\frac{\text{Number of correct components}}{7} \right. \\ & \left. + \frac{2 - \text{Number of incorrect components}}{7} \right) \\ & + 3 * \frac{\text{Number of correct relationships}}{9} \\ & + 2 * \frac{\text{Number of correct directions}}{9} \end{aligned}$$

for the COMP solution. The score varies from a minimum of 0 to a maximum of 10.

The higher weights for classes/components than those for relationships can be justified based on the fact that the subjects could reuse the classes and components available in the libraries which, however, do not store any relationships. It is therefore appropriate to penalize subjects more for not selecting a correct class/component than for not identifying a correct relationship. Also, given that correct and incorrect classes/components were both available in the libraries, the scoring formula considers the failure to identify a correct class/component to have the same effect as that of selecting an incorrect one.

Note that for any analysis task, more than one correct solution is possible. For example, **ClaimIntimation** is shown as an association class in Fig. 1, derived from the many-to-many relationship between **Claim** and **Claimant**. Instead of modeling it as an association class, one could make **ClaimIntimation** a regular class and create two one-to-many relationships, one between **Claim** and **ClaimIntimation**, and the other between **Claimant** and **ClaimIntimation**. The solution evaluation process we followed explicitly recognized the possibility of such differences and made sure that all correct modeling alternatives were scored equally.

7. Experimental results

In the first experimental task, participants were asked to list the relevant objects/components needed to support the requirements described in the business case provided to them. Overall, the subjects correctly selected the needed assets (objects or components) 84.2% of the time. The 37 OO subjects selected the required classes only 68.8% of the time, however. In contrast, each of the 36 COMP subjects was able to identify all the components needed for modeling the system. We compared the selection rates for the two groups of subjects using the non-parametric Mann-Whitney test (Siegel 1956) and found that COMP subjects were significantly better in selecting the required components than were the OO subjects in selecting the required classes (Mann-Whitney $U = 72$, $p = 0.000$).

Table 4 presents the means and standard deviations of the performance scores—objective as well as subjective—grouped by the representation used, for the conceptual modeling task. All the COMP scores, both objective and subjective, are much higher than the corresponding OO scores.

Table 4
Descriptive statistics for performance scores by group.

Measure	OO			COMP		
	Mean	Standard deviation	N	Mean	Standard deviation	N
Obj	4.996	1.674	37	7.681	0.968	36
CM Sub	5.095	1.896	37	6.639	1.234	36

Table 5
ANCOVA results for representation.

Variables	CMObj (N = 73)			CMSub (N = 73)		
	df	F	Significance	df	F	Significance
Corrected model	5	16.040	.000	5	5.653	.000
Representation	1	62.518	.000	1	14.844	.000
SA&D Experience	1	4.491	.038	1	5.833	.018
OOA&D Experience	1	.672	.415	1	.838	.363
Prog. Experience	1	.007	.934	1	.281	.598
OO Prog. Experience	1	2.147	.148	1	3.820	.055

7.1. Inter-rater reliabilities

We determined the inter-rater reliability between the two raters using the two-way mixed effects model in SPSS, where subject effects are random and rater factor is fixed; the measure for inter-rater reliability is the intraclass correlation coefficient (McGraw and Wong 1996). For each subject solution, the mean of the subjective scores assigned by the two raters—which typically produces more reliable measurements—was used for analyzing the results. For that reason, we report the average measures for the intraclass correlation coefficient. The intraclass coefficient for the subjective scores assigned by the two raters is 0.916, significantly greater than 0 (no consistency) at the $p < 0.001$ level. The coefficient is higher than 0.80, indicating that there is a high degree of reliability between the scores assigned by the two raters.

7.2. Subjective and objective score reliabilities

To assess the reliability between the subjective and objective measurements, we computed the intraclass correlation coefficients for the *Obj* and *Sub* scores. As with the two raters, we employed the two-way mixed effects models, and defined agreement in terms of the consistency between measurements. But unlike last time, we report the single measures coefficient, as opposed to average measures, because our goal is not to combine the two ratings, but to use them individually. The intraclass correlation coefficient between *CMObj* and *CMSub* is 0.874; it is significantly greater than 0 (no consistency) at the $p < 0.001$ level. There is a high degree of consistency between the two sets of measurements, reinforcing the validity of our scoring schemes.

7.3. Conceptual modeling performance results

We used ANCOVA to test for the differences between the performance scores of the OO and COMP groups (Hypothesis H1). Because subjects were randomly assigned to the two groups, the main objective of using covariance in this study was to reduce within group or error variance (Stevens 1992). Within group variability could arise because of potential covariates relating to analysis/design and programming experience.

We conducted the ANCOVA test using the GLM (General Linear Model) Univariate procedure in SPSS. The GLM Univariate procedure provides regression analysis and analysis of variance for one dependent variable by one or more factors and/or variables. The factor variables divide the population into groups. We used one

dependent variable (Performance) for each analysis of variance, and one factor (Representation). The four covariates mentioned above were used as predictor variables for each regression analysis. Table 5 presents the results of the ANCOVA.

The ANCOVA results for both *CMObj* and *CMSub* showed a significant influence of Representation ($p = 0.000$). These results strongly support Hypothesis H1, which states that the conceptual models analysts develop by selecting and assembling components are of higher accuracy than those they develop by selecting and assembling objects. Among the covariates, SA&D Experience had a significant effect on performance for both *CMObj* and *CMSub* ($p = 0.038$ and $p = 0.018$, respectively), and OO Programming Experience was marginally significant for *CMSub* ($p = 0.055$).

7.4. Effects of SA&D experience

As noted above, SA&D Experience was found to be related to the CM scores. To explore the effect of experience individually for each representation, we conducted separate multiple regressions. For each regression, performance (*CMObj* or *CMSub*) is the dependent variable, and the independent variables are SA&D Experience, OOAD Experience, Programming Experience, and OO Programming Experience. The regression results for the OO representation and COMP representation are shown in Tables 6 and 7, respectively.

Table 6(a) and (c) show that the ANOVAs for the OO representation are significant for *CMObj* and *CMSub*, respectively. Also, Table 6(b) and (d) show that SA&D Experience has a significant influence over both *CMObj* ($p = 0.038$) and *CMSub* ($p = 0.018$). These results strongly support Hypothesis H2, which states that experienced systems analysts develop more accurate conceptual models of business systems by reusing objects than those developed by less experienced analysts.

In contrast, Table 7(a) and (c) present a different picture. The ANOVAs for the COMP representation are not significant ($p = 0.415$ and $p = 0.428$ for *CMObj* and *CMSub*, respectively). Also, Table 7(b) and (d) show that the effects of SA&D Experience are not significant ($p = 0.309$ and $p = 0.234$ for *CMObj* and *CMSub*, respectively). These results support Hypothesis H3, which states that experienced analysts will not have an advantage over less experienced analysts in developing conceptual models through component reuse.

We conducted further analyses by categorizing SA&D Experience into two groups, *Low* and *High*, with the *Low* group having values of 1 and 2, and the *High* group having values greater than

Table 6
Regression results for OO.

(a) Analysis of Variance for CMObj					
	Sum of squares	Df	Mean square	F ratio	Significance
Regression	21.537	4	5.384	2.171	.095
Residual	79.345	32	2.480		
(b) Variables in Equation for CMObj					
Variables	Regression coefficient	Standard error of coefficient	Standardized regression coefficient (β)	t value	Significance
Constant	3.191	1.285		2.483	.018
SA&D Experience	.716	.330	.426	2.170	.038
OOA&D Experience	-.100	.387	-.048	-.257	.799
Prog. Experience	-.295	.387	-.138	-.762	.452
OO Prog. Experience	.578	.374	.262	1.545	.132
(c) Analysis of Variance for CMSub					
	Sum of squares	Df	Mean square	F ratio	Significance
Regression	33.271	4	8.318	2.768	.044
Residual	96.148	32	3.005		
(d) Variables in Equation for CMSub					
Variables	Regression coefficient	Standard error of coefficient	Standardized regression coefficient (β)	t value	Significance
Constant	3.415	1.415		2.414	.022
SA&D Experience	.908	.363	.477	2.499	.018
OOA&D Experience	-.184	.426	-.078	-.432	.669
Prog. Experience	-.566	.426	-.234	-1.329	.193
OO Prog. Experience	.787	.412	.315	1.910	.065

Table 7
Regression results for COMP.

(a) Analysis of Variance for CMObj					
	Sum of squares	df	Mean square	F ratio	Significance
Regression	3.794	4	.948	1.015	.415
Residual	28.977	31	.935		
(b) Variables in Equation for CMObj					
Variables	Regression coefficient	Standard error of coefficient	Standardized regression coefficient (β)	t value	Significance
Constant	7.595	.826		9.191	.000
SA&D Experience	.199	.192	.202	1.034	.309
OOA&D Experience	-.640	.328	-.402	-1.951	.060
Prog. Experience	.030	.171	.031	.175	.862
OO Prog. Experience	.204	.197	.195	1.038	.307
(c) Analysis of Variance for CMSub					
	Sum of squares	df	Mean square	F ratio	Significance
Regression	6.036	4	1.509	.990	.428
Residual	47.269	31	1.525		
(d) Variables in Equation for CMSub					
Variables	Regression coefficient	Standard error of coefficient	Standardized regression coefficient (β)	t value	Significance
Constant	6.108	1.055		5.787	.000
SA&D Experience	.298	.246	.237	1.213	.234
OOA&D Experience	-.706	.419	-.348	-1.687	.102
Prog. Experience	.015	.219	.012	.070	.945
OO Prog. Experience	.365	.251	.274	1.456	.156

2. Note that the mean for SA&D Experience for the entire pool of subjects is 2.49, so the cutoff for categorization is reasonable.

The mean CM scores for OO were higher for subjects with high SA&D experience (CMObj = 5.714, CMSub = 5.861) compared to those with low experience (CMObj = 4.316, CMSub = 4.368). For the COMP group, however, subjects with high SA&D experience (CMObj = 7.649, CMSub = 6.595) did not perform better than those with low experience (CMObj = 7.726, CMSub = 6.700).

To explore the joint effects of all the factors and variables, we adopted a 2 × 2 factorial design, with Representation and SA&D

Experience Level as the two factors, each having two values. Using the SPSS GLM procedure, we conducted an ANCOVA for the CM score (separately for objective and subjective scores) with the two factors and the three covariates: OOA&D Experience, Programming Experience, and OO Programming Experience.

For CMObj, the main effects for Representation and SA&D Experience Level were both significant ($p = 0.000$ and $p = 0.020$, respectively), and so was the interaction effect between Representation and SA&D Level ($p = 0.020$). For CMSub, the analysis revealed similar results. The main effects for Representation and

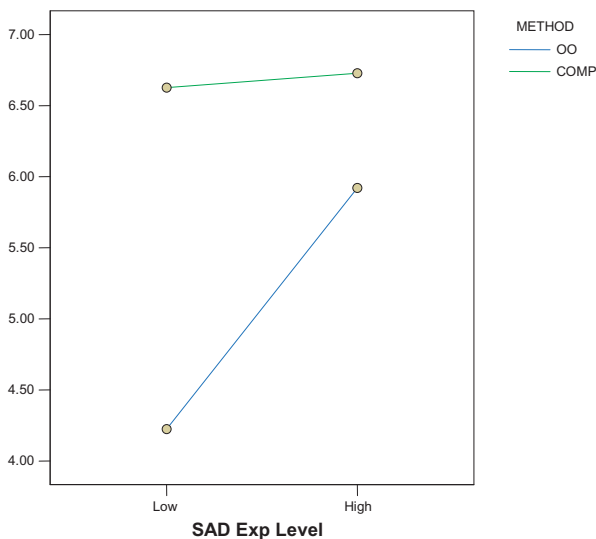


Fig. 6. Interaction effects between SA&D exp level and representation.

SA&D Experience Level were both significant ($p = 0.000$ and $p = 0.038$, respectively), and the interaction effect was also significant ($p = 0.033$).

The interaction effects between Representation and SA&D Experience Level—after accounting for the effects of the covariates—are shown graphically in Fig. 6 for CMSub (the graph for CMObj are similar). In the figure, the CMSub score is plotted against SA&D Experience Level separately for each representation. The bottom line represents the OO representation and the top line represents the COMP representation. We can see that while there is a major difference in performance between the two experience groups for OO, the performance remains relatively flat between the two experience groups for COMP, resulting in different slopes for the two lines. The line representing OO has a much higher slope than the line representing COMP. The interaction effects between Representation and SA&D Experience Level can therefore be explained by the difference in slopes of the two lines.

7.5. Threats to validity

In empirical software engineering research, it is important to minimize the threats to the validity of the research findings. In this section, we discuss how we addressed different types of validity threats, including those to construct validity, internal validity, and external validity.

Construct validity is concerned with whether a test measures the intended construct. Threats to construct validity could arise from the choice of treatment and the choice of outcome measure. The choice of the treatment involves operationalization of the independent variable and the administration of the treatment, and the choice of outcome measure involves operationalization of the dependent variable and the administration of the measurement.

We conducted a pilot study to test construct validity before running the actual experiment. Two subjects used OOAD and two subjects used CBD to complete the experimental tasks, after which they were debriefed. They were satisfied with the training and the experimental materials; based on the suggestion from one of the participants, we included a practice exercise that subjects in the actual experiment completed prior to participating in the actual experiment.

In this study the antecedent (independent variable) is the representation provided by the method used, which is either *object* or *component*. Because the participants were divided into

two groups, with one group asked to model a business system by reusing objects, and the other group asked to do the same by reusing components, there was no threat to construct validity with respect to the operationalization of the independent variable. Also, when we presented the treatments to the participants, they had no clue about the results we were expecting from the experiment; the administration of the treatments, therefore, did not pose any threat to construct validity.

The outcome measure of interest is *Performance*, which indicates how well a subject modeled the business system by reusing objects or components. As discussed in detail above, we measured performance using two scores: objective (CMObj) and subjective (CMSub). The formula for the objective score was based on existing knowledge in conceptual modeling, while the subjective score, which reflects the overall quality of the developed model, was assigned independently by two experts. As reported before, the intraclass correlation coefficient between CMObj and CMSub was quite high, suggesting strongly the validity of our scoring schemes. Also, the inter-rater reliability between the two experts on the subjective score was very high, further strengthening the construct validity of our study.

We tested our hypotheses using a well-designed, controlled experiment where any systematic bias—a source of threat to internal validity—was effectively addressed by randomly assigning subjects to the two groups. The desired effects of randomization were achieved; the two groups were similar with respect to the demographic/experience attributes. We also took precautions against any other plausible explanation for the study's findings by including covariates (SA&D Experience, OOA&D Experience, Programming Experience, OO Programming Experience, and Insurance Knowledge), which factored out any differential effects they may have on the dependent variable.

Threats to external validity can arise from non-representative sampling or a non-representative research context. One criticism that can be levelled against our research is that the grain size of the components was much larger than that of the classes used in the study. Because of that, the number of components was much less than the number of classes in the libraries that participants used. However, both libraries capture the same insurance claim domain completely, so they are informationally equivalent. Additionally, these libraries were built by an IT firm for use in the insurance domain, and can be considered to be representative of situations in the real world. From a realistic standpoint, therefore, it does not make sense to artificially alter the size or number of classes/components. We therefore believe that the research findings of our study are generalizable to similar research contexts where libraries have been built, based on sound design principles, for industrial use. To minimize the threat to external validity, we also used a representative sample of IT professionals in the experiment.

8. Discussion

By empirically demonstrating the influence of modeling representation and systems analysis experience on reuse-based conceptual modeling performance, this study fills a major gap in the literature. We designed an experiment to test three theory-driven hypotheses. It is imperative that theory and empirical studies in software engineering interact so that we can acquire practical knowledge (Hannay et al. 2007). We now discuss the practical implications of our research.

An important implication of our research findings is that organizations that maintain component repositories can reuse the stored components effectively. For organizations that do not maintain such repositories, a viable and attractive option would

be to buy off-the-shelf components from third-party vendors than to develop systems in-house from scratch.

We found that even those IT professionals who did not have much experience in traditional systems analysis and design performed on par with experienced professionals when modeling business systems using components that represent discrete business functions. A major implication of this finding is that firms do not have to invest a lot of time and money in training entry-level analysts at using CBD for conceptual modeling. This study would encourage firms—especially those that are facing major attrition problems—to adopt the CBD approach. Also, with the growing interest in web services, firms could weigh in the potential for developing composite or higher-level web services by assembling components.

Components originated in the implementation environment as run-time binary units that could be executed. However, as discussed before, their use has gradually spread to the realm of analysis and design through the concept of business components. This is similar to the origins of OOAD two decades ago, which was preceded by and rooted in OO programming languages. In this study, one of our goals was to examine the reuse effectiveness of the two representational constructs for modeling business systems. We found that COMP modelers outperformed OO modelers in selecting and assembling the reusable assets. This is an important finding given that prior research comparing OO and process-oriented (PO) tools has found OO tools lagging behind when it comes to modeling behavior (Agarwal et al. 1996a), while their performance level was more or less the same for structure-oriented tasks.

Both the class and component libraries used for this experimental study were built for use by an IT firm for applications in the insurance domain. The subjects who participated in the experiment therefore used libraries that were geared toward industrial use. These libraries can be considered to be typical of what one would expect to find in the real world. Therefore, increasing or decreasing the grain size of a class/component is not an option we could afford. The classes and components used in this study were fabricated based on sound design principles.

9. Limitations of the study

In this section, we discuss the limitations of our study. As discussed before, the components used in the study were fabricated by a global IT firm for use in the insurance claims domain. The components were designed, using classes from existing class libraries, by maximizing cohesion and minimizing coupling of the classes within each component and optimizing objectives such as ease of assembly, maintainability, and cost effectiveness. Clearly, the development of such components from the underlying classes entails substantial effort, something that we did not include in our comparison of the two approaches. However, note that component development for a specific domain incurs only a one-time cost, and if the components are reused in multiple applications, the effects of that cost peter out over time.

There is a good chance that an IT consulting firm may be contracted to develop a business system in a domain for which it does not have an existing component library. One option for the firm would be to buy off-the-shelf components from a third party, plug them together, and deliver the solution. Clearly, the firm needs to factor the acquisition costs into the cost-benefit equation, but as long as these costs are much lower than those for in-house development, this approach would be a more pragmatic and efficient option than developing the system from scratch.

Because the focus of our study was on modeling business systems through reuse, any conclusions we draw will have to be confined within that context. For instance, we cannot make any statements regarding the efficacy of either of the two modeling

approaches for developing business systems from scratch. Note, however, that because of the increasing availability of business components, we do not expect this to be a major limitation. For example, SAP has exposed most of its core functionality through web-service interfaces. Also, IBM has developed business components for various domains.

In situations where components are not available, even for purchase from third-party vendors—for example, when the domain is relatively unstructured or not well understood—the firm may be forced to develop the business system from scratch. In such situations, where the requirements are new, developing business systems from objects may be more effective, given that they provide more flexibility than components. Also, depending on the type of components available, it may not be possible to tailor the components to satisfy requirements of custom systems (Carney and Long 2000). Future studies, therefore, could delve into comparative evaluations that address class and component design issues in the face of new or customized requirements.

In our study, the number of components and the relationships among them was much less than that of the classes and their relationships. That happened because the grain size of the components used in the study was much larger than that of the objects. As noted above, the class and component libraries were built by an IT firm and are representative of what they actually look like in real-world business applications. Therefore, we did not have the freedom to change the size of the components. Also, higher grain size alone does not necessarily imply better modeling performance. The components also need to be cohesive enough, otherwise they will carry a lot of baggage, making them more difficult and less likely to be reused during service integration.

10. Conclusions and future directions

In this study, we conducted an empirical investigation into the effectiveness of the object and component representations for modeling business systems. Our work should be viewed within the context of software reuse research because the participants in this study modeled systems by reusing objects/components from an existing library. While several proponents have argued about the benefits of CBD, our study addresses a major gap in the literature by demonstrating, using a well-controlled experiment, that component-based reuse is indeed more effective than object-oriented reuse for modeling business systems. All the participants were IT professionals working for a global IT consulting firm, so the usual criticisms leveled against the use of novices or student subjects do not apply. The evaluation of the two approaches was also very comprehensive in that the experimental tasks included modeling a system from the perspective of selecting the right assets and assembling those assets. Prior research in cognitive psychology and human factors provided the theoretical underpinnings for our study.

The most interesting finding of our study is the interaction between chunk representation and experience. As hypothesized in our research model, we found the presence of moderating effects of systems analysis and design experience on the relationship between systems development method and conceptual modeling performance. The service-oriented abstraction provided by components and their interfaces allows a less experienced analyst to perform on par with more experienced analysts.

The participants in this study had very little or no knowledge of the automobile insurance domain. An interesting future direction would be to examine if knowledge of the application domain has any bearing on modeling performance. Shaft and Vessey (1995) conducted a study in the area of program comprehension and found that programmers familiar with the domain use a hierarchical top-down comprehension process, as opposed

to a bottom-up inferential process for unfamiliar domains. In a more recent study, [Khatri et al. \(2006\)](#) found that application domain knowledge influences the performance on conceptual schema-based problem solving tasks for which cognitive fit does not exist. The interesting finding of their study is that application domain knowledge improves problem-solving performance only for demanding problem-solving tasks, but not for comprehension tasks where cognitive fit exists.

Future research could explore the effects of domain knowledge on conceptual modeling tasks, using objects and components. An interesting direction to pursue is to investigate if domain knowledge could close the performance gap between objects and components for modeling tasks. Problem solvers who have a lot of domain knowledge may possess superior knowledge structures that they could exploit. It is possible that domain experts would be able to recognize larger, meaningful chunks of objects, and by so doing try to neutralize the relative advantage offered by components. Note, however, that it is unrealistic to assume that IT professionals will have sound knowledge in every application domain they work in and, within that context, our study does provide compelling evidence in favor of CBD for modeling business systems through reuse.

In addition to application domain knowledge, [Khatri et al. \(2006\)](#) also examined the effects of IS domain knowledge. They found that IS knowledge improves performance in all types of conceptual schema understanding tasks, both in familiar and unfamiliar domains. More IS domain knowledge translates to more experience in systems analysis and design in this study. The interesting finding in our study is that the influence of systems development method on modeling performance is contingent upon systems analysis and design experience. While more experienced analysts are better at reusing objects for business modeling, they do not perform better than less experienced analysts when reusing components. Given that the subjects had very little or zero experience in OOAD and CBD, it would be interesting to examine if OOAD and CBD experience influences modeling performance, and if so, do those two types of experience differentially affect performance?

[Burton-Jones and Meso \(2006\)](#) examined if UML analysis diagrams adhering to the good decomposition principles of minimality, determinism, losslessness, weak coupling, and strong cohesion improve analysts' understanding of the domain. Their study, which focuses on OO analysis models, could be extended to the CBD space. The decomposition principles mentioned above are also very important for component design. Future research could therefore explore if component models based on good decompositions are better in terms of understanding than those that are not.

[Parsons and Saunders \(2004\)](#) conducted empirical studies investigating the influence of cognitive heuristics on artifact reuse. Specifically, they considered *anchoring*, which results when problem solvers tend to anchor solutions to new problems on reusable artifacts and adjust them. Components, by definition, are sealed and encapsulated, implying that they cannot be modified. However, it would be interesting to examine the anchoring behavior of analysts reusing objects and components for modeling business problems that have requirements which are slightly different from the functionality captured explicitly by those reusable artifacts.

Another interesting research direction would be in the area of model comprehension. Getting end users involved at all stages is imperative for rapid systems development. To ensure that users participate actively in the development process and provide appropriate inputs and feedback, it is necessary for the analysts and designers to effectively communicate the models they build to the users. Systems development methods could therefore be compared based on how well end users are able to comprehend the artifacts (models) they generate. For instance, prior research has examined if end users find process-oriented models easier or more diffi-

cult to comprehend than object-oriented models ([Agarwal et al. 1999](#)). Similarly, we could compare component-based models with object-oriented models from a model comprehension standpoint.

Finally, our research expands upon prior research examining performance in conceptual modeling by providing a multi-dimensional approach to characterizing system decomposition. The three dimensions we used to characterize a representational construct—decomposition granularity, decomposition quality, and focus—provide us with a much more comprehensive approach than has been possible hitherto for comparing the merits/complexities of different constructs. Prior research has typically compared conceptual modeling along one of the axes. For example, while some studies have used “decomposition quality” as the basis for evaluation (e.g., [Burton-Jones and Meso 2006, 2008](#)), other studies—such as the ones on cognitive fit (e.g., [Agarwal et al. 1996a; Vessey and Galletta 1991](#))—have used the “focus” of the problem representation used. But the flexibility afforded by the *component* construct allows us to conduct evaluations along the three axes simultaneously, opening up interesting possibilities for future research.

Components and objects can be viewed as belonging to a continuum of constructs in this three-dimensional space. Varying its position along these axes helps determine a construct's decomposition characteristics and complexity, and ultimately its effect on reuse-based modeling performance. In addition, performance will also depend on the nature of the *mindshift* ([Armstrong and Hardgrave 2007](#)) that the analyst must undergo in transitioning to a new representational construct in this space. The transition from structured systems analysis and design to OOAD was considered to be an architectural change ([Sircar et al. 2001](#)). However, given that the transition from objects to components takes place along a continuum, it is expected to introduce evolutionary, rather than radical, changes in the environment.

Acknowledgments

We thank the guest editors of this special issue and three anonymous referees for their constructive feedback. We also want to thank Ajoy Mukherjee, Narayana Mandaleeka, and Kushal Banerjee of Tata Consultancy Services for helping us in conducting the experiments.

Appendix A. Experimental materials

Business case: ABC's Claims Processing System

ABC Insurance Co. sells automobile insurance policies to customers as a safeguard against damages from accidents, acts of vandalism, arson, thefts, etc. When such an incident takes place, ABC Insurance is notified and provided with the necessary information, including details of the following: the person filing the claim; any victims involved in the incident; the nature and site of the incident; any damages to properties such as cars and buildings; and any other party involved in the incident (e.g., driver or passenger of the other car). All this information is entered into the claims processing system. A claim handler retrieves the stored information and checks all claim-related details. The handler uses the external policy system to retrieve details of the insurance policy and its coverage limits. Next, he or she requests supporting documents, validates the claim based on those documents, and registers the claim. The claim handler also opens a reserve account for paying all claim-related expenses.

ABC then identifies a licensed assessor and assigns him or her with the responsibility of estimating the correct amount of the claim. The estimate is prepared based on factors such as severity of injuries, type and price of property damaged, description of damages, etc. After the estimate is prepared, the status of the claim

and the reserve amount are updated, and the assessor is paid for providing the services; the payment is treated as a claim expense.

Given below are three scenarios that provide examples for auto insurance claim situations.

Scenario A

Julie Huston had parked her car at the parking lot next to the Giant Eagle grocery store in the Monroeville suburb of Pittsburgh on January 12. After finishing her shopping, she returned to the parking lot and found that someone had broken into her car and stolen the car's CD player. The window on the driver's side was broken. Julie called ABC and informed her of the incident; she received an estimate of \$330 for getting the window replaced and \$400 for getting a new CD player installed.

Scenario B

Bill Smith was driving home on the night of February 12 and the roads were slippery with ice. As he turned his car right onto River Street in Naperville, a suburb of Chicago, it swerved and hit the mailbox in front of the first house on the right. Although Bill was not injured, the front right door of his car was heavily dented and the mailbox was totally damaged. The owner of the house, Sean Weston, called ABC and informed that it would cost \$150 to have a new mailbox installed. ABC was also informed by Bill of a quote of \$450 that he had received from Pete's Body Repair Shop for repairing the car door.

Scenario C

Jane Marsh was involved in an accident on July 26 when her car hit a pickup truck from behind. This happened near the Cincinnati downtown exit on Interstate 75. The tail lights and the rear of the pickup truck were badly damaged. The driver of the truck did not suffer any injuries but Jane suffered neck injuries. The front bumper of her car was damaged. Jane visited the doctor's office the same day and received initial treatment for her neck injuries; she had to pay a fee of \$70 for the visit. Jane later called ABC Insurance and informed an insurance representative of the accident, the doctor's fee, and the damage to her car's bumper (approximately \$200 to fix). After taking down the information, the representative validated and registered the claim. Because Jane appeared to be at fault, the driver of the pickup truck called ABC and claimed damages to his truck (estimated to be \$650) on Jane's insurance policy. Dave Williams was assigned to estimate the different amounts for the claim. Dave prepared the estimates based on the available information and updated the claim status. ABC paid Dave \$200 for his services and this amount was recorded as a claim expense.

Appendix B. Class library

Part of Claims System Class Report

Claim

Claim is a request for compensation for the loss suffered against a particular policy. Following are the different types of claim:

- 1 Body Injury claim
- 2 Auto Damage Claim
- 3 Property/Structure Damage Claim.

Operations

- updateClaimStatus()

Changes the status of claim viz., Intimated / Validated / Assessed / Rejected / Settled etc. Various claim sub processes like claim validation, assessment, investigation, arbitration and case will update the claim status.

- validateClaim()

Validates the intimated claim for the correctness of the policy information, the coverage details (whether the claimed amount is with-in the policy limits), and the correctness of the supporting documents that are provided.

- checkMandatoryClaimDetails()

Verifies the claim being intimated with the company's business rules (that are signed during the policy purchase) viz., that the loss event must have taken place with-in a geographic area and the registered driver must have been driving the car during the loss event.

- checkDuplicateClaim()

Checks whether a claim with the same information has already been created to avoid redundancy.

- intimateClaimRejection()

If the claim validation process results in failure, then the reason for the claim rejection is intimated to the Claimant.

- checkEventIntimationDetails()

Verifies the Intimation details like the claimed amount, the addresses provided by the clamant about the loss event, the current location of the property/vehicle involved, etc.

- createClaimRecord()

Creates a claim record after the validation process resulting in success, and intimating the re-insurer / co-insurer (if necessary), and requesting the claimant for any additional supporting documents.

- getClaimDetails()
- Retrieves the claim details.
- intimateCoInsurer()

If the policy was purchased with a Co-Insurance Option, then the Co-Insurer is intimated about the claim.

- intimateReInsurer()

If the policy was purchased with a Re-Insurance Option, then the Re-Insurer is intimated about the claim.

Parent Classes

None

Claimant

Claimant is a person/company who opens a claim against a policy. A claimant can be the insured person, any authorized representative, third party suffering a loss due to the insured's fault, or the insurance company of the suffered third party. One claimant can have many claims.

Operations

- setRelationshipWithInsured()

Used to modify the relationship with the insured person.

- addClaimantDetails()

Gets and stores personal details of claimant including his/her relationship with the insured.

Parent Classes

Party

ClaimExpense

For every claim, at every stage, a reserve amount is maintained. The decision on the reserve amount is at the discretion of the Insurance Company. The reserve amount for a claim should not exceed the sum insured. This is used by the accounts department to find out the total reserves at any given point of time.

Operations

- getTotalAmountSpent()

Retrieves the total amount spent on a specific claims process of the complete claim itself.

- getAmountSpentforPartyId()

Retrieves the expenses incurred by a specific person.

- recordClaimExpense()

Records various claim expenses with relevant description of the expense like the nature of expenses.

Parent Classes
None

Appendix C. Component library

Part of Claims System Component Report

Claim Initiation

The intimation information about a claim as described by the intimator is captured and processed by this component. This component will make necessary calls to the external policy system to retrieve and validate the policyholder's details in order to accept the intimation. This component also makes necessary calls to the other components to store explicit information about the loss event, and the properties and persons involved in the loss event

Component interfaces

- I_Policy

Operations

- getPolicyDetails ()

This service makes calls to an external policy system to check existence of valid policy and retrieve the policy information and the policyholder's information

- getCoverLimitDetails ()

This service will call the external policy system to access the information about the "Limits" for a "Coverage" in order to facilitate the service invoker to justify whether the claimed amount is with-in the limits or not.

- I_Intimation

Operations

- initiateIntimation ()

This is an interactive service that is triggered by claimant or a representative on his behalf. This service prompts user for the information and updates various records like the claimant estimates. In order to add the details of the claimant, loss event, and involved party, this service makes necessary calls to the services provided by other components.

- getIntimationDetails ()

This service makes necessary calls to other components to access the personal, property and loss event information and combines with the loss estimate information, to return intimation details. These details are used to verify the correctness of the data provided by the claimant and register a claim.

- changeClaimantEstimates ()

This service offers a mechanism to update the loss estimates during various phases of the claim processing by the authorized users.

Claim Registration

This component validates the information provided by the claimant and registers a Claim. A claim is a request for compensation for the loss suffered against a particular policy. This component handles two different types of claims: 1) Auto Damage Claim and 2) Property/Structure Damage Claim In order to process the information about the Body Injury Claim, this component makes necessary calls to the other components. A reserve amount is maintained to cover estimated loss and misc expenses. The decision on the reserve amount is at the discretion of the Insurance Company. This component maintains a log/record of the expenses incurred towards claims processing governed by certain business rules like limitation of expenses towards a particular sub process.. This provides services to find out the total expenses at any given point of time, and services to record the expenses that are spent as fees or miscellaneous expenses This component also processes necessary documents (FIR, DL, MVI Report etc) which are required for processing the claim and raises a request if a business user needs it.

Component interfaces

- I_ClaimRegistration

Operations

- validateClaim ()

The service obtains the necessary intimation information from other components, and validates them against the correctness of policy, supporting documents etc. After validating the intimation, this service registers it as a valid claim and updates the claim status.

- getClaimStatus ()

This service retrieves the current claim status needed by various actors involved in the claims process, in order to ensure smooth business flow.

- getClaimDetails ()

This service retrieves all the information about a "Validated Claim" so that the authorized claim handlers to assess it further can access this.

- I_ClaimAccounts&Finance

Operations

■ submitSupportingDocuments ()

This service provides an uploading mechanism, where the invoker submits the supporting documents, and its headers so that the service will organize and maintain them efficiently.

■ requestSupportingDocuments ()

This service sends a message / mail to the party asking him to submit necessary documents.

■ recordClaimExpense ()

This service records various claim expenses with relevant description of the expense like the nature of expenses.

References

- Adelson, B., 1981. Problem solving and the development of abstract categories in programming languages. *Memory Cognition* 9, 422–433.
- Agarwal, R., De, P., Sinha, A.P., 1999. Comprehending object and process models: an empirical study. *IEEE Trans. Softw. Eng.* 25 (4), 541–556.
- Agarwal, R., Sinha, A.P., Tanniru, M., 1996a. Cognitive fit in requirements modeling: a study of object and process methodologies. *J. Manage. Inf. Syst.* 13 (2), 137–162.
- Agarwal, R., Sinha, A.P., Tanniru, M., 1996b. The role of prior experience and task characteristics in object-oriented modeling: an empirical study of object and process methodologies. *Int. J. Hum. Comput. Stud.* 45, 639–667.
- Armstrong, D.J., Hardgrave, B.C., 2007. Understanding mindshift learning: the transition to object-oriented development. *MIS Quarterly* 31 (3), 453–474.
- Barbier, F., Atkinson, C., 2003. Business component. In: Barbier, F. (Ed.), *Business Component-Based Software Engineering*. Springer, pp. 1–26.
- Birkmeier, D., Overhage, S., 2009. On component identification approaches – classification, state of the art, and comparison. In: Lewis, G.A., Poernomo, I., Hofmeister, C. (Eds.), *Component Based Software Engineering*. Springer, Berlin, pp. 1–18.
- Booch, G., 1994. *Object-Oriented Analysis and Design with Applications*, second ed. Benjamin/Cummings, Redwood City, CA.
- Booch, G., Rumbaugh, J., Jacobson, I., 1999. *The Unified Modeling Language User Guide*. Addison-Wesley, MA Reading.
- Brown, W., 2000. *Large-Scale, Component-Based Development*. Prentice Hall, Upper Saddle River, NJ.
- Burton-Jones, A., Meso, P.N., 2006. Conceptualizing schemas for understanding: an empirical test of decomposition principles in object-oriented analysis. *Inf. Syst. Res.* 17 (1), 38–60.
- Burton-Jones, A., Meso, P.N., 2008. The effects of decomposition quality and multiple forms of information on novices' understanding of a domain from a conceptual model. *J. Assoc. Inf. Syst.* 9 (12), 748–802.
- Buxton, W., 1986. Chunking and phrasing and the design of human-computer dialogues. In: *Proc. IFIP World Computer Congress*. Dublin, Ireland, pp. 475–480.
- Carney, D., Long, F., 2000. What do you mean by COTS? Finally, a useful answer. *IEEE Softw.* 17 (2), 83–86.
- Chang, S.H., Han, M.J., Kim, S.D., 2005. A tool to automate component clustering and identification. In: *Proc. 8th Int'l Conf. on Fundamental Approaches to Software Eng.*, pp. 141–144.
- Chase, W.G., Simon, H.A., 1973. Perception in chess. *Cognitive Psychol.* 4 (1), 55–81.
- Chi, M.T.H., Glaser, R., Rees, E., 1982. Expertise in problem solving. In: Sternberg, R.J. (Ed.), *Advances in the Psychology of Human Intelligence*. Erlbaum, Hillsdale, NJ 1.
- Crnkovic, I., Stafford, J., Szyperski, C., 2011. Software components beyond programming: from routines to services. *IEEE Softw.* 28 (3), 22–26.
- de Cesare, S., Lycett, M., Macredie, R.D., 2006. Development of component-based information systems: an introduction. In: de Cesare, S., Lycett, M., Macredie, R.D. (Eds.), *Development of Component-Based Information Systems: Advances in Management Information Systems*. M. E. Sharpe, Armonk, NY, pp. 3–20.
- Due, R.T., 2000. The wisdom of patterns. *Cutter IT J.* 13 (8), 6–9.
- Elfatry, A., 2007. Dealing with change: components versus services. *Comm. ACM.* 50 (8), 35–39.
- Erl, T., 2004. *Service-Oriented Architecture: A Field Guide to Integrating XML and Web Services*. Prentice Hall, Upper Saddle River, NJ.
- Erl, T., Karmarkar, A., Walmsley, P., Haas, H., Yalcinalp, U., Liu, C.K., Orchard, D., Tost, A., Pasley, J., 2009. *Web Service Contract Design & Versioning*. Prentice Hall, Upper Saddle River, NJ.
- Frakes, W.B., Kang, K., 2005. Software reuse research: status and future. *IEEE Trans. Softw. Eng.* 31 (7), 529–536.
- Gane, C., Sarson, T., 1979. *Structured Systems Analysis: Tools and Techniques*. Prentice Hall, Englewood Cliffs, NJ.
- Haines, M.N., Rothenberger, M.A., 2010. How a service-oriented architecture may change the software development process. *Comm. ACM.* 53 (8), 135–140.
- Hannay, J.E., Sjøberg, D.I.K., Dybå, T., 2007. A systematic review of theory use in software engineering experiments. *IEEE Trans. Softw. Eng.* 33 (2), 87–107.
- Heineman, G.T., Councill, W.T., 2001. *Component Based Software Engineering: Putting the Pieces Together*. Addison-Wesley, Boston, MA.
- Herzum, P., Sims, O., 2000. *Business Component Factory: A Comprehensive Overview of Component-Based Development for the Enterprise*. John Wiley, Hoboken, NJ.
- IBM, 2016. *IBM Knowledge Center: WebSphere Commerce Version 7*. http://www.ibm.com/support/knowledgecenter/SSZL2C-7.0.0/landing/wc_welcome.html.
- Jain, H., Zhao, H., Chinta, N.R., 2007. Web services identification: methodology and CASE tools. In: Zhang, L.J. (Ed.), *Modern Technologies in Web Services Research*. IGI Global, New York, NY.
- Jang, Y.J., Kim, E.Y., Lee, K.W., 2003. Object-oriented component identification method using the affinity analysis technique. *Lect. Notes Comput. Sci.* 2817, 317–321.
- Janssen, M., Wagenaar, R.W., 2006. Business engineering of component-based systems. In: de Cesare, S., Lycett, M., Macredie, R.D. (Eds.), *Development of Component-Based Information Systems: Advances in Management Information Systems*. M. E. Sharpe, Armonk, NY, pp. 166–181.
- Jonkers, H., Lankhorst, M.M., Quartel, D.A.C., Proper, E., Iacob, M.-E., 2011. ArchiMate(R) for integrated modelling throughout the architecture development and implementation cycle. In: *Proc. 13th Conf. on Commerce and Enterprise Computing*. IEEE, pp. 294–301.
- Kerlinger, F.N., 1986. *Foundations of Behavioral Research*, third ed. Holt, Rinehart and Winston, New York, NY.
- Khatiri, V., Vessey, I., Ramesh, V., Clay, P., Park, S., 2006. Understanding conceptual schemas: exploring the role of application and IS domain knowledge. *Inf. Syst. Res.* 17 (1), 81–99.
- Koschmider, A., Fellmann, M., Schoknecht, A., Oberweis, A., 2014. Analysis of process model reuse: where are we now, where should we go from here. *Decis. Support Syst.* 66, 9–19.
- LaFrance, M., 1989. The quality of expertise: implications of expert-novice differences for knowledge acquisition. *SIGART Newsletter* 108, 6–14 Knowledge Acquisition Special Issue.
- Larkin, J., McDermott, J., Simon, D., Simon, H., 1980. Expert and novice performance in solving physics problems. *Science* 208, 140–156.
- Larkin, J.H., Simon, H.A., 1987. Why a diagram is (sometimes) worth ten thousand words. *Cognitive Sci.* 11, 65–99.
- Larman, C., 2002. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*, second ed. Prentice Hall, Upper Saddle River, NJ.
- Lau, K.-K., Wang, Z., 2007. Software component models. *IEEE Trans. Softw. Eng.* 33 (10), 709–724.
- Lee, J.K., Jung, S.J., Kim, S.D., Jang, W.H., Ham, D.H., 2001. Component identification method with coupling and cohesion. In: *Proc. 8th Asia-Pacific Software Eng. Conf.*. IEEE Computer Society Press.
- Levi, K., Arsanjani, A., 2002. A goal-driven approach to enterprise component identification and specification. *Comm. ACM.* 45 (10), 45–52.
- Lorenz, H., Vliissides, J., 2001. Designing components versus objects: a transformational approach. In: *Proc. 23rd Int'l Conf. on Software Eng.* Toronto, Canada, pp. 253–262.
- Mahmood, S., Ahmed, M., Alshayeb, M., 2014. Analysis and evaluation of software artifact reuse environments. *Int. J. Softw. Innovation.* 2 (2), 54–65.
- McGraw, K.O., Wong, S.P., 1996. Forming inferences about some intraclass correlation coefficients. *Psychol. Methods* 1 (1), 30–46.
- McKeithen, K.B., Reitman, J.S., Rueter, H.H., Hirtle, S.C., 1981. Knowledge organization and skill differences in computer programmers. *Cognitive Psychol.* 13, 307–325.
- Papert, S., 1980. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, NY.
- Parsons, J., Saunders, C., 2004. Cognitive heuristics in software engineering: applying and extending anchoring and adjustment to artifact reuse. *IEEE Trans. Softw. Eng.* 30 (12), 873–888.
- Schneider, J., Han, J., 2004. Components – the past, the present, and the future. In: *Proc. Ninth Int'l Workshop on Component-Oriented Programming*. Oslo, Norway.
- Shaft, T.M., Vessey, I., 1995. The relevance of application domain knowledge: the case of computer program comprehension. *Inf. Syst. Res.* 6 (3), 286–299.
- Shneiderman, B., 1976. Exploratory experiments in programmer behavior. *Int. J. Comput. Inf. Sci.* 5, 123–143.
- Siegel, S., 1956. *Nonparametric Statistics for the Behavioral Sciences*. McGraw-Hill, New York, NY.
- Sinha, A.P., Jain, H., 2013. Ease of Reuse: An Empirical Comparison of Components and Objects. *IEEE Software* 30 (5), 70–75.
- Sircar, S., Nerur, S.P., Mahapatra, R., 2001. Revolution or evolution? A comparison of object-oriented and structured systems development methods. *MIS Quarterly* 25 (4), 457–471.
- Sprott, D., 2000. Enterprise resource planning: componentizing the enterprise application packages. *Comm. ACM.* 43 (4), 63–69.
- Stevens, J., 1992. *Applied Multivariate Statistics for the Social Sciences*. Lawrence Erlbaum, Hillsdale, NJ second ed.
- Sugumar, V., Storey, V.C., 2003. A semantic-based approach to component retrieval. *DATA Base Adv. Inf. Syst.* 34 (3), 8–24.
- Szyperski, C., 2002. *Component Software: Beyond Object-Oriented Programming*. Addison-Wesley, Boston, MA second ed.
- Vessey, I., 1991. Cognitive fit: a theory-based analysis of the graphs versus tables literature. *Decis. Sci.* 22 (2), 219–240.
- Vessey, I., Galletta, D., 1991. An empirical study of information acquisition. *Inf. Syst. Res.* 2 (1), 63–84.

- Vitharana, P., Jain, H., 2000. Research issues in testing business components. *Inf. Manage* 37 (6), 297–309.
- Vitharana, P., Jain, H., Zahedi, F.M., 2003a. Design, retrieval, and assembly in component-based software development. *Comm. ACM* 46 (11), 97–102.
- Vitharana, P., Jain, H., Zahedi, F.M., 2004. Strategy-based design of reusable business components. *IEEE Trans. Syst. Man Cybern. Part C* 34 (4), 460–474.
- Vitharana, P., Jain, H., Zahedi, F.M., 2012. A knowledge based component/service repository to enhance analysts' domain knowledge for requirements analysis. *Inf. Manage* 49, 24–35.
- Vitharana, P., Zahedi, F.M., Jain, H., 2003b. Knowledge-based repository scheme for storing and retrieving business components: a theoretical design and an empirical analysis. *IEEE Trans. Softw. Eng.* 29 (7), 649–664.
- Wand, Y., Weber, R., 1990. An ontological model of an information system. *IEEE Trans. Softw. Eng.* 16 (11), 1282–1292.
- Wood, R.E., 1986. Task complexity: definition of the construct. *Organizational Behav. Hum. Decis. Process* 37 (1), 60–82.
- Zhang, L.J., 2007. *Modern Technologies in Web Services Research*. IGI Global, NewYork, NY.
- Zimmermann, O., Schlimm, N., Waller, G.N., Pestel, M., 2005. Analysis and design techniques for service-oriented development and integration. IBM Deutschland, Stuttgart, Germany, pp. 1–6.

Atish P. Sinha is Professor of Information Technology Management and Director of the Center for Technology Innovation at the Lubar School of Business, University of Wisconsin-Milwaukee. He earned his Ph.D. in business, with a concentration in Artificial Intelligence, from the University of Pittsburgh. His current research interests are in the areas of business intelligence and analytics, data mining, text mining, service-oriented computing, and healthcare informatics. His research has been published in several journals, including *Communications of the ACM*; *Decision Support Systems*; *IEEE Transactions on Software Engineering*; *IEEE Transactions on Systems, Man, and Cybernetics*; *Information Systems Research*; *International Journal of Human-Computer Studies*; *Journal of the Association for Information Systems*; *Journal of Biomedical Informatics*; and *Journal of Management Information Systems*. He currently serves as a Senior Editor for *DATA BASE for Advances in Information Systems*.

Hemant Jain is W. Max Finley Chair for Excellence in American Business: Data Analytics in the College of Business at the University of Tennessee Chattanooga. Before that, he was Professor of Information Technology Management at the University of Wisconsin-Milwaukee. He received his Ph. D. in Information Systems from Lehigh University, M. Tech. from IIT Kharagpur, India, and B. E. from University of Indore, India. Dr. Jain specializes in service-oriented architecture, component-based development, real-time enterprises, data analytics, text mining and healthcare informatics. Dr. Jain is on the steering committee of the IEEE technical committee on service computing. He also serves on the editorial board of a number of highly regarded Journals. His work has been published in journals such as *MIS Quarterly*, *ISR*, *JMIS*, *JAIS*, *EJIS*, and *IEEE Transactions on Software Engineering*.